

SOFTWARE TESTING

Curriculum



SOFTWARE TESTING

JAVA

1. Datatype

- Primitive Data Types (int, char, boolean, etc.)
- Non>Primitive Data Types (String, Arrays, Classes, etc.)
- Type Casting (Widening and Narrowing)

2. OOPS (Object>Oriented Programming)

- Class and Object Encapsulation
- Inheritance Polymorphism
- Abstraction

3. Inheritance

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance (through interfaces)
- Method Overriding

4. Polymorphism

- Compile>time Polymorphism (Method Overloading)
- Runtime Polymorphism (Method Overriding)

5. Overloading

- Method Overloading
- Constructor Overloading
- Rules for Overloading (changing parameters, return type, etc.)

6. Overriding

- Method Overriding
- Rules for Overriding (access level, return type, etc.)
- Super Keyword
- Final Keyword in Overriding

7. Abstraction

- Abstract Classes
- Interfaces (in relation to abstraction)

8. Interface

- Default Methods in Interfaces
- Static Methods in Interfaces
- Multiple Inheritance using Interfaces

9. Constructor > Constructor Chaining

- Default Constructor > Parameterized Constructor
- Constructor Chaining (within same class using `this()`)

10. Conditional Statement (if else)

- Simple `if` statement
- `if>else` statement
- `else if` ladder
- Nested `if>else` statements
- Ternary Operator

11. Control Statement (for loop)

- `for` Loop > `while` Loop > `do>while` Loop
- Enhanced `for` Loop (`for>each`) > Break and Continue Statements

12. Scanner Class

- Creating a Scanner Object
- Reading Different Data Types (nextInt(), nextLine(), nextDouble(), 13. Variable
- Local Variables > Instance
- Variables > Static Variables
- etc.)

14. Switch Casting

- Switch Statement (Syntax, Rules)

15. Array

- Array Initialization
- Array Length and Iteration
- (Possibly meant to be "Type Casting" and "Switch Statement")

16. String > String Class

- String Methods (`length()`, `charAt()`, `substring()`, etc.)
- String iteration > StringBuilder and StringBuffer (MutableStrings)

17. Collection 1

- List, Set > List Interface (ArrayList, LinkedList)
- Set Interface (HashSet, TreeSet, LinkedHashSet)
- Differences between List and Set > Common Operations (Add, Remove, Iterate)

18. Collection 2

- Map > Map Interface (HashMap, TreeMap, LinkedHashMap) > Key>Value Pairs
- Iterating through a Map > Common Map Methods (`put()`, `get()`, `containsKey()`, etc.)

19. Iterator

- Iterator Interface (hasNext(), next(), remove())
- ListIterator > For>each Loop vs. Iterator

20. Exception

- Types of Exceptions (Checked vs. Unchecked)
- Common Exceptions (NullPointerException, ArrayIndexOutOfBoundsException, etc.)
- Creating Custom Exceptions > Exception Propagation

21. Exception Handling

- Try>Catch Block
- Finally Block
- Throw and Throws Keyword
- Multiple Catch Blocks

SELENIUM

1. Browser Launch

- Setting up WebDriver > BrowserDriver Executables (chrome driver)
- Opening URLs with ``get()``` and ``navigate().to()```
- Maximising the Browser Window Closing the Browser (``close()``` vs. ``quit()```)

2. WebDriver Methods

- Navigation Commands (``navigate().back()```, ``navigate().forward()```, ``refresh()```)
- Page Interaction Commands (``getTitle()```, ``getCurrentUrl()```, ``getPageSource()```)

3. WebDriver Elements

- Finding Elements

(`findElement()`, `findElements()`) > Interacting with Elements (`click()`, `sendKeys()`, `clear()`, `submit()`) > Retrieving Element Attributes (`getAttribute()`, `getText()`, `getCssValue()`) > Handling Element States (`isDisplayed()`, `isEnabled()`, `isSelected()`)

4. Locators

> ID Locator > Name Locator > Class Name
> Locator > Tag Name Locator > Link Text and
> Partial Link Text Locator > CSS Selector
> XPath (Absolute vs. Relative XPath)

5. Drop Down

- Selecting Options using `Select`
Class (`selectByVisibleText()`, `selectByValue()`, `selectByIndex()`) Deselecting Options
(`deselectAll()`, `deselectByVisibleText()`, etc.)
- Handling Multi-Select Drop-downs Retrieving All Options
(`getOptions()`, `getAllSelectedOptions()`,
`getFirstSelectedOption()`)

6. Take Screenshot

- Capturing Full Page Screenshot
(`TakesScreenshot` Interface) Capturing Web
- Element Screenshot
- Capturing Full Page Screenshot(using Utility File
> Apache)

7. Alert

- Handling JavaScript Alerts (`alert.accept()`,
`alert.dismiss()`)
- Handling Confirmation Boxes > Handling
Prompt Boxes (`sendKeys()` in Alerts)
- Switching to Alert (`switchTo().alert()`) >
Retrieving Alert Text (`getText()`)

8. Frame

- Identifying Frames
(`findElement(By.tagName("iframe"))`)
- Switching between Frames
- (`switchTo().frame()`,
`switchTo().defaultContent()`)
- Nested Frames Handling > Switching Back to
Main Content

9. Drag Down

- Scrolling Web Pages (using JavaScriptExecutor)
- Scrolling to Specific Elements > Drag and Drop Actions (`Actions` Class)
- Vertical and Horizontal Scrolling

10. Keyboard Actions

- Keyboard Operations (`Actions` Class > `sendKeys()`, `keyDown()`, `keyUp()`)
- Handling Key Combinations (e.g., Ctrl+C, Ctrl+V)
- Simulating Keyboard Input to Elements
- Sending Special Keys (e.g., ENTER, TAB)

11. Windows Handling

- Handling Multiple Browser Windows (`getWindowHandles()`, `switchTo().window()`)
- Switching Between Windows and Tabs > Closing Specific Windows

12. Wait

- Implicit Wait > Explicit Wait (`WebDriverWait` and `ExpectedConditions`)
- Fluent Wait (Polling intervals and timeout)

PROJECT CLASS

1.Base Class

- Centralised WebDriver setup and teardown
- Project Class
- Common utility methods (e.g., screenshots, click>)
- Reusable test configurations

2.POJO

- Simple Java objects with getters and setters
- Used for data representation in tests
- Supports serialisation/deserialisation

3.POM (Page Object Model)

- Separates page elements and actions into page classes
- Enhances code maintainability and reusability
- Reduces code duplication

4.Data Driven

- Tests driven by external data (Excel)
- Parameterisation using TestNG/JUnit
- Apache POI for Excel integration

5.TestNG

- Annotations (@Test, @BeforeMethod, etc.)
- Test grouping, prioritization, and parallel execution
- Data Providers for parameterization
- 6.Singleton Design Pattern
- Ensures a single instance of a class
- Commonly used for WebDriver management

7.Cucumber BDD

- Gherkin syntax (Given, When, Then) Feature files and step definitions
- Cucumber Hooks (@Before, @After) JUnit
- Annotations (@Test, @Before, @After)
Assertions for test validation
- Integration with Maven/Gradle

8. Error Handling

- Managing API errors and exceptions
- Validating error response

9. Test Runner

Playwright + Cypress+ TypeScript + Framework

UI & API Automation tool

- Report tracking
- Mocha reports
- Allure reports

Module 1: TypeScript Basics

Topics

- Introduction to TypeScript
- Installing Node.js, npm, and TypeScript > TypeScript setup (tsconfig.json)
- Data types: string, number, boolean, any, void
- Variables: let, const
- Operators and expressions
- Conditional statements (if, switch)
- Loops (for, while, for>of)

Practical Exercises:

- Implement calculator operations using TypeScript
- Write programs using loops and conditions
- Print multiplication tables, sum of arrays, etc.

Module 2: TypeScript Functions and OOP Basics

Topics:

- Function declaration, anonymous functions, arrow functions
- Optional and default parameters
- Rest parameters (...args)
- Introduction to Classes & Objects
- Constructor and methods
- Access modifiers: public, private, protected
- Inheritance, Polymorphism
- Interfaces and Generics

Practical Exercises:

- Create a BankAccount class with deposit/withdraw methods
- Implement inheritance: SavingsAccount & CurrentAccount
- Use interfaces for multiple account types

Module 3: Advanced TypeScript Concepts

Topics:

- Type assertions and type guards

- Enums and tuples
- Union & intersection types
- Namespaces and modules
- Error handling with try>catch>finally > Async/Await with Promises > Importing/exporting modules

Practical Exercises:

- Fetch data asynchronously using Promises and Async/Await
- Write a module for reusable utility functions
- Handle exceptions in user input validation

Module 4: DOM & Events with TypeScript

Topics:

- TypeScript with HTML DOM
- Selecting elements (getElementById, querySelector)
- Handling events (click, change, submit)
- Event delegation
- Manipulating DOM elements dynamically
- Forms and input validation

Module 5: Playwright Basics

Topics:

- Introduction to Playwright
- Playwright vs Selenium
- Installing Playwright and dependencies > Browser launching (Chromium, Firefox, WebKit)
- Page navigation & interactions
- Locators: CSS, XPath, Text, Role
- Synchronisation: waits, timeouts, and assertions

Practical Exercises:

- Automate login/logout on a demo website
- Navigate multiple pages and validate titles
- Handle basic alerts and popups

Module 6: Playwright Advanced

Topics:

- Handling multiple tabs and iFrames
- Mouse and keyboard events
- Drag>and>drop and sliders
- Shadow DOM handling
- Capturing screenshots, videos, and traces
- Parameterized testing and loops in tests

- Parallel test execution

Practical Exercises:

- Automate a product purchase flow on an e-commerce site
- Capture screenshots and videos for validation
- Run tests on multiple browsers in parallel

Module 7: Playwright API Testing

Topics:

- Introduction to API testing with Playwright
- HTTP methods: GET, POST, PUT, DELETE
- Sending requests and validating responses > Handling authentication (token, headers)
- API mocking and network interception
- Integrating API calls into UI tests

Practical Exercises:

- Send GET request to fetch user data
- Send POST request to create a new resource
- Validate response status, headers, and payload
- Mock an API response and verify frontend updates

Module 7: Playwright API Testing

Topics:

- Introduction to API testing with Playwright
- HTTP methods: GET, POST, PUT, DELETE
- Sending requests and validating responses > Handling authentication (token, headers)
- API mocking and network interception
- Integrating API calls into UI tests

Practical Exercises:

- Send GET request to fetch user data
- Send POST request to create a new resource
- Validate response status, headers, and payload
- Mock an API response and verify frontend updates

Module 8: Framework with Playwright (Cucumber + Data>Driven)

Topics:

- Introduction to BDD & Cucumber
- Gherkin syntax: Feature, Scenario, Given>When>Then
- Integrating Cucumber with Playwright

- Creating feature files and step definitions > Hooks (Before, After, BeforeStep, AfterStep)
- Tags for scenario execution
- Reporting in Cucumber

Practical Exercises:

- Automate login scenario using Cucumber feature file
- Execute selected scenarios using tags > Generate HTML/Cucumber reports

Module 9: Data>Driven Framework

Topics:

- Concept of data>driven testing
- Reading test data from JSON, CSV, Excel
- Parameterizing Playwright tests
- Combining data>driven testing with Cucumber
- Reusable functions and utilities
- Best practices for maintainable framework

Practical Exercises:

- Run login tests with multiple usernames/ passwords from JSON
- Automate product search using CSV data input

- Build a reusable function for form submission using Excel data

Benefits for Students:

- Starts from TypeScript fundamentals builds to Playwright automation
- Includes UI + API testing > Covers framework best practices (Cucumber + data>driven)
- Hands>on real>time practical exercises after every module
- Ready for real>world projects & interviews

SQL

1. Database Basics

- Understanding databases, tables, and schemas
- Key concepts: Primary Key, Foreign Key, Indexes

2. SQL Queries

- Writing basic queries: `SELECT`, `INSERT`, `UPDATE`, `DELETE`
- Using `WHERE`, `ORDER BY`, `GROUP BY`

3. Joins

- Understanding and using different types of joins (INNER, LEFT, RIGHT, FULL)
- Joining multiple tables

4. Aggregate Functions

- Using `COUNT()`, `SUM()`, `AVG()`, `MAX()`,
`MIN()`
- Grouping data with `GROUP BY` and
`HAVING`

5. Subqueries

- Writing nested queries
- Using subqueries in `SELECT`, `WHERE`, and
`FROM` clauses

6. Database Management

- Creating and altering tables
- Managing users and permissions
- Backup and restore operations



urbancode.in